

目录



- [01、Plan-and-Execute 的核...](#)
- [02、任务建模](#)
 - [为什么需要任务建模](#)
 - [Task 类设计](#)
 - [任务的生命周期](#)
 - [依赖关系](#)
 - [依赖关系](#)
 - [执行计划](#)
 - [拓扑排序算法](#)
 - [计划状态管理](#)
- [03、规划器实现](#)
 - [规划提示词工程](#)
 - [解析 LLM 输出](#)
 - [重新规划](#)
- [04、PlanExecuteAgent](#)
 - [智能模式切换](#)
- [05、计划可视化](#)
- [06、运行测试](#)
- [07、和 ReAct 的对比](#)
 - [什么时候用 ReAct](#)
 - [什么时候用 Plan-and-Execute](#)
 - [混合使用](#)
- [08、进阶：并行执行](#)
 - [并行执行可能遇到的问题](#)
 - [更智能的规划](#)
 - [规划的自我修正](#)
- [PaiCLI 如何写到简历上？](#)

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)

Search

原创

给Agent CLI加上Plan-and-Execute，让Agent先规划后执行，支持DAG。

大家好，我是二哥呀。

PaiCLI 的第 1 期我们已经实现了，一个基础的 ReAct Agent，能一步一步执行任务，一边思考一边行动。

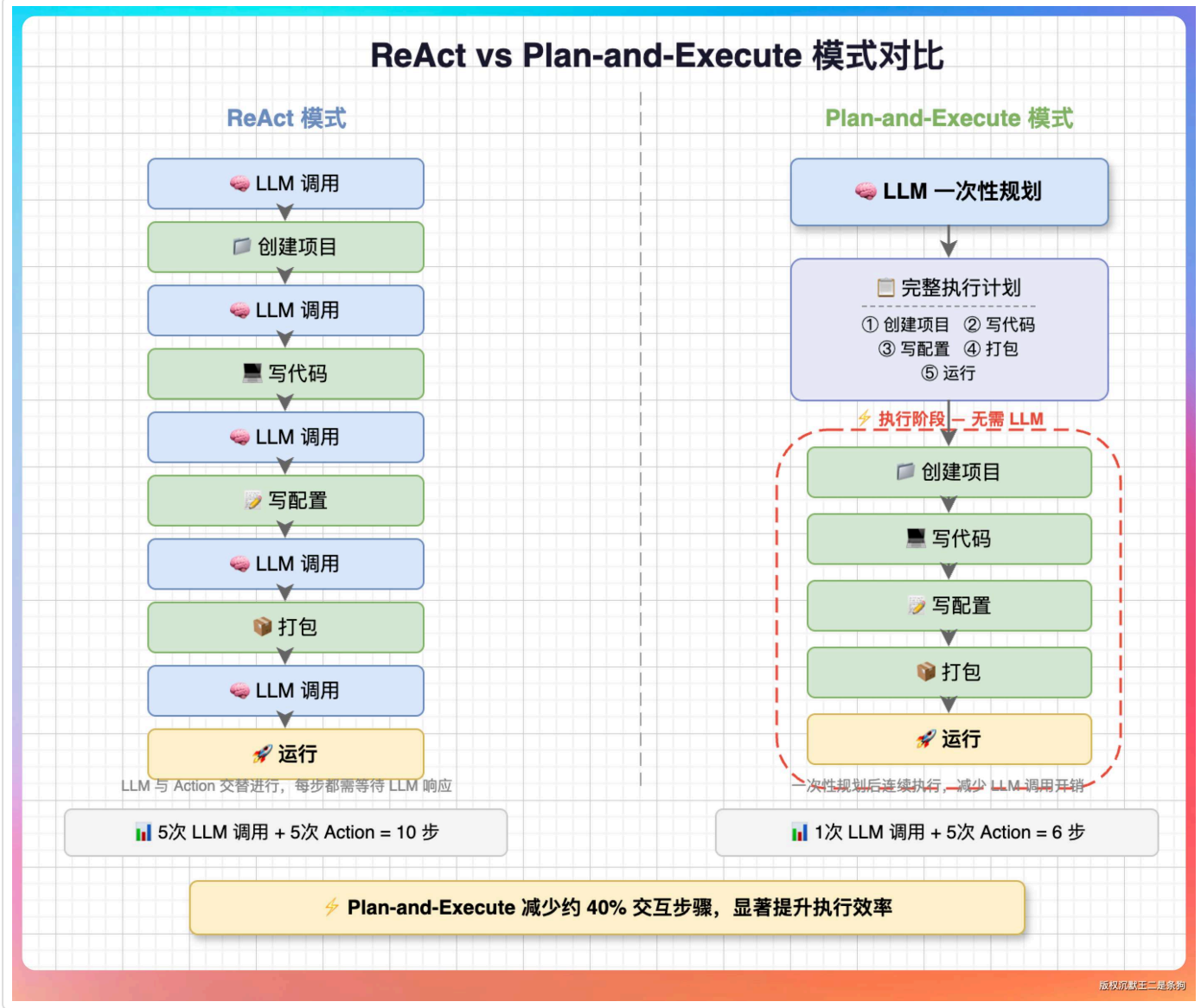
但这种方式有个问题：**复杂任务需要很多轮对话**，每一步都需要调用 LLM。

比如“创建一个 Spring Boot 项目，写个 REST API，然后打包运行”这个任务：

第 2 期，我们来实现 **Plan-and-Execute** 模式：先让 LLM 制定完整计划，然后按步骤执行，中间不再反复询问 LLM。

Plan-and-Execute 模式来自论文《Plan-and-Solve Prompting》。

PaiCLI 如何写到简历上?



任务类型我们定义了 6 种：

- **PLANNING**：规划任务，用于分析和决策
- **FILE_READ**：读取文件，获取信息
- **FILE_WRITE**：写入文件，输出结果
- **COMMAND**：执行命令，编译运行等
- **ANALYSIS**：分析结果，中间决策
- **VERIFICATION**：验证结果，检查正确性

任务状态有 5 种：

- **PENDING**：等待执行
- **RUNNING**：执行中
- **COMPLETED**：已完成
- **FAILED**：执行失败
- **SKIPPED**：被跳过（依赖失败）

任务的生命周期

一个任务从创建到完成，完整的生命周期如下所示：

PENDING → RUNNING → COMPLETED/FAILED/**SKIPPED**

objectivec 复制代码

每个状态转换都有对应的方法：

java 复制代码

```
public void markStarted() {
    this.status = TaskStatus.RUNNING;
    this.startTime = System.currentTimeMillis();
}

public void markCompleted(String result) {
    this.status = TaskStatus.COMPLETED;
    this.result = result;
    this.endTime = System.currentTimeMillis();
}

public void markFailed(String error) {
    this.status = TaskStatus.FAILED;
    this.error = error;
    this.endTime = System.currentTimeMillis();
}
```

记录时间戳有两个用途：一是统计执行耗时，二是分析任务瓶颈。如果某个任务总是耗时很长，可能需要优化或者拆分。

依赖关系

复杂任务有先后依赖。比如“写代码”依赖“创建项目”，“运行”依赖“编译”。

我们用 DAG（有向无环图）表示依赖关系：

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

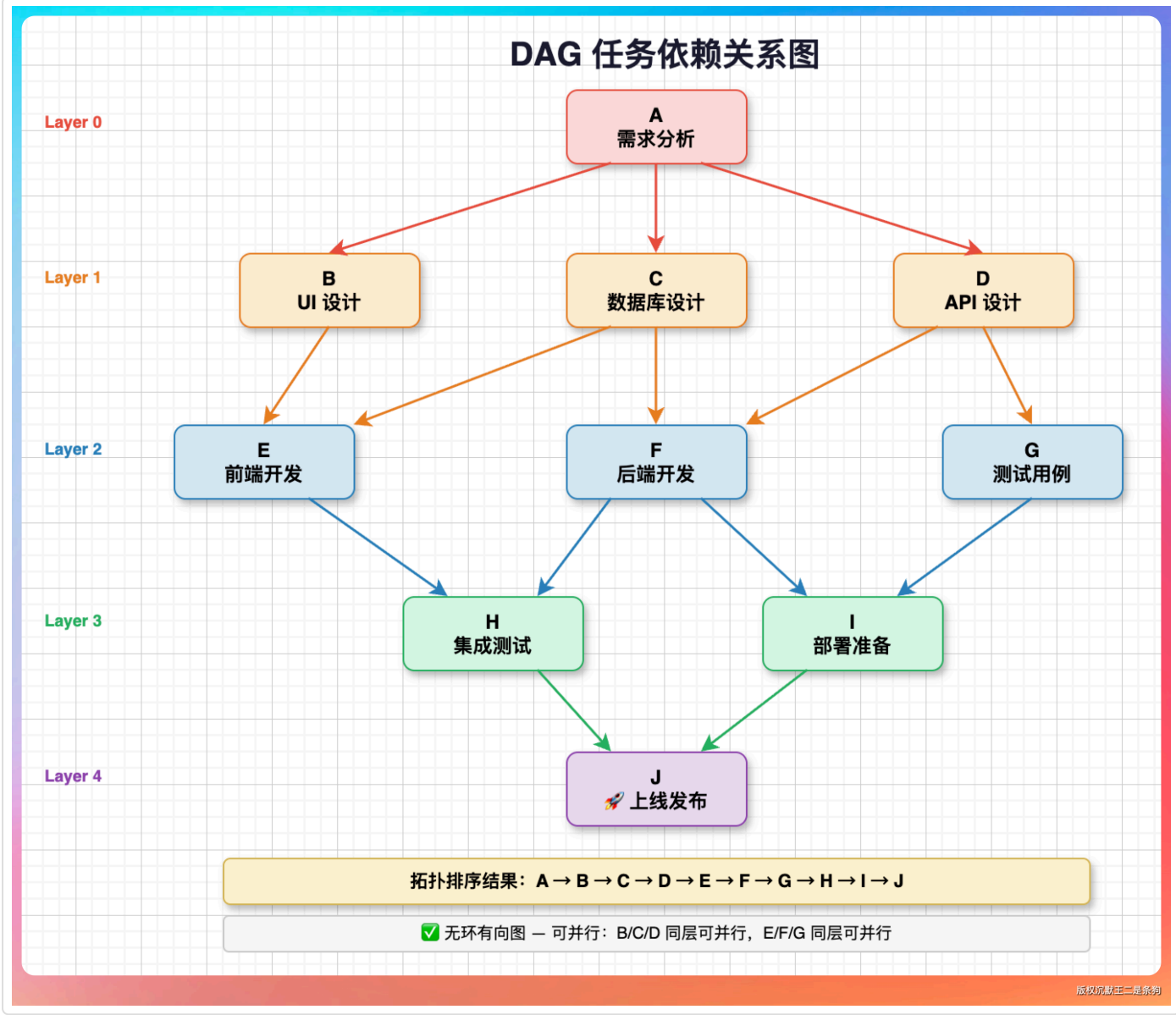
[08、进阶：并行执行](#)

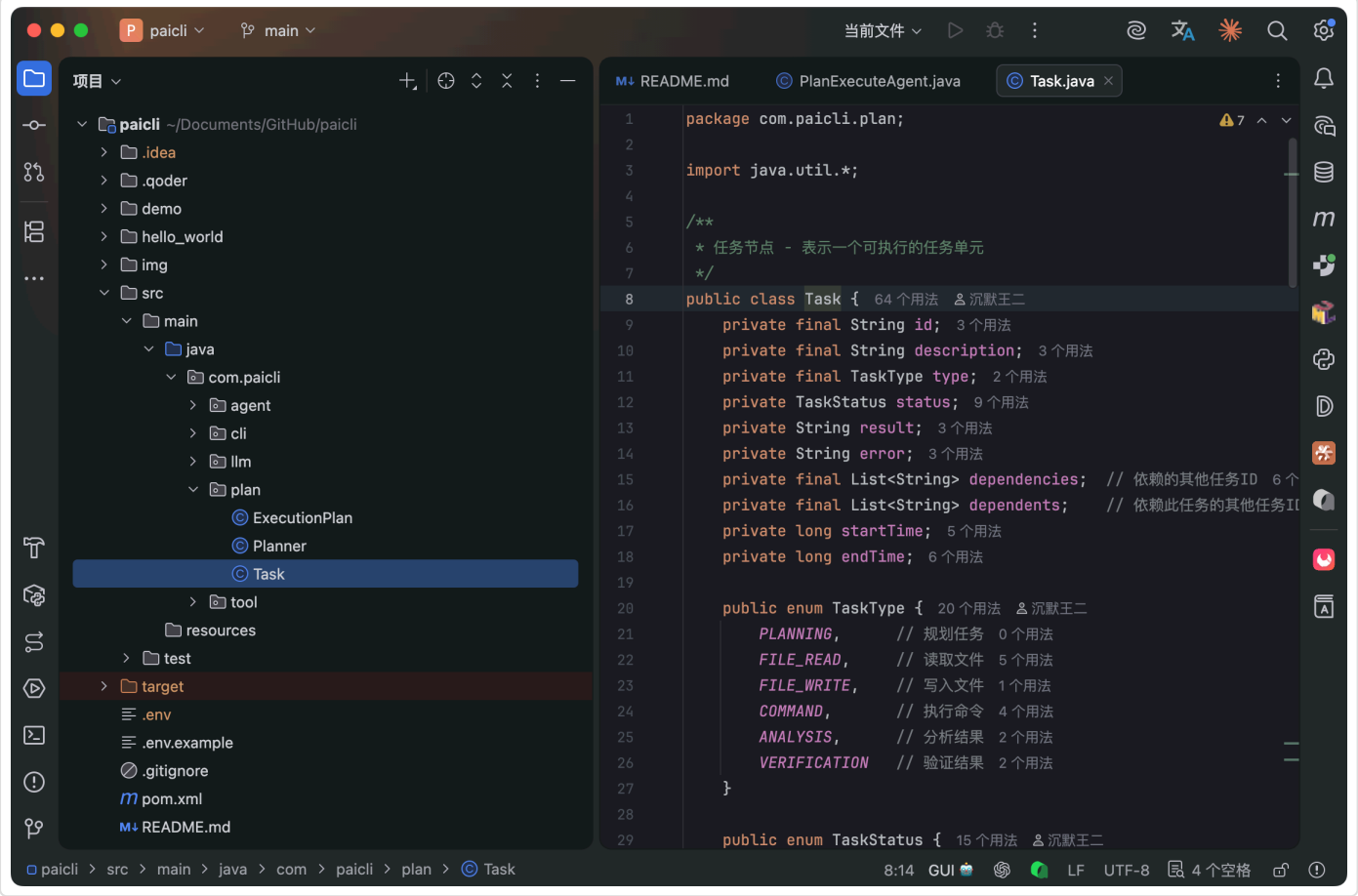
[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)

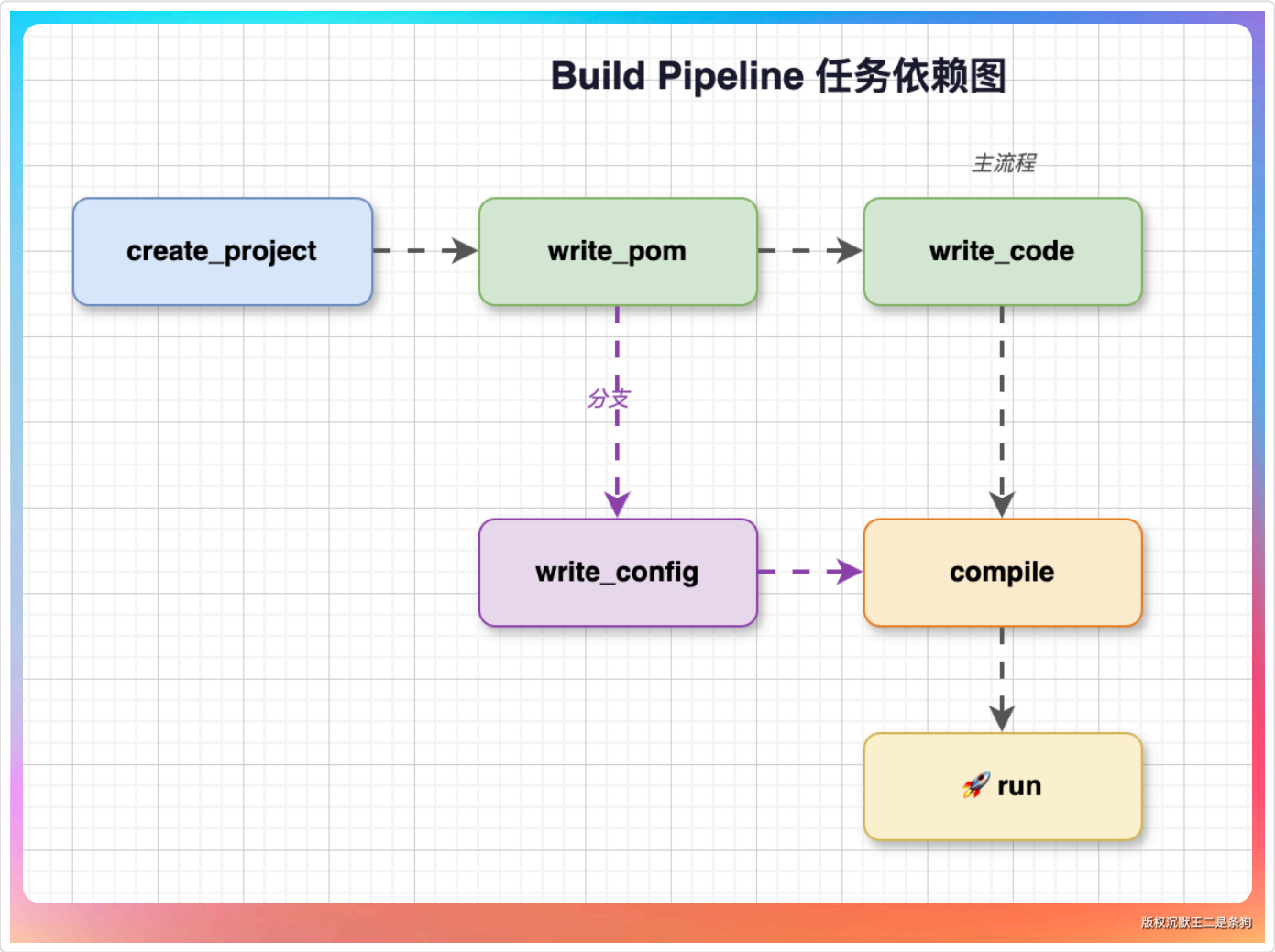




依赖关系

复杂任务有先后依赖。比如“写代码”依赖“创建项目”，“运行”依赖“编译”。

我们用 DAG（有向无环图）来表示依赖关系：



每个任务可以声明自己依赖哪些任务（dependencies），系统会自动计算出执行顺序。

执行计划

多个任务可以组成一个执行计划：

```
public class ExecutionPlan {
    private final String id;
    private final String goal;           // 计划目标
    private final Map<String, Task> tasks; // 所有任务
    private final List<String> executionOrder; // 执行顺序
    private PlanStatus status;
    private String summary;
}
```

java 复制代码

目录

01、Plan-and-Execute 的核...

02、任务建模

为什么需要任务建模

Task 类设计

任务的生命周期

依赖关系

依赖关系

执行计划

拓扑排序算法

计划状态管理

03、规划器实现

规划提示词工程

解析 LLM 输出

重新规划

04、PlanExecuteAgent

智能模式切换

05、计划可视化

06、运行测试

07、和 ReAct 的对比

什么时候用 ReAct

什么时候用 Plan-and-Execute

混合使用

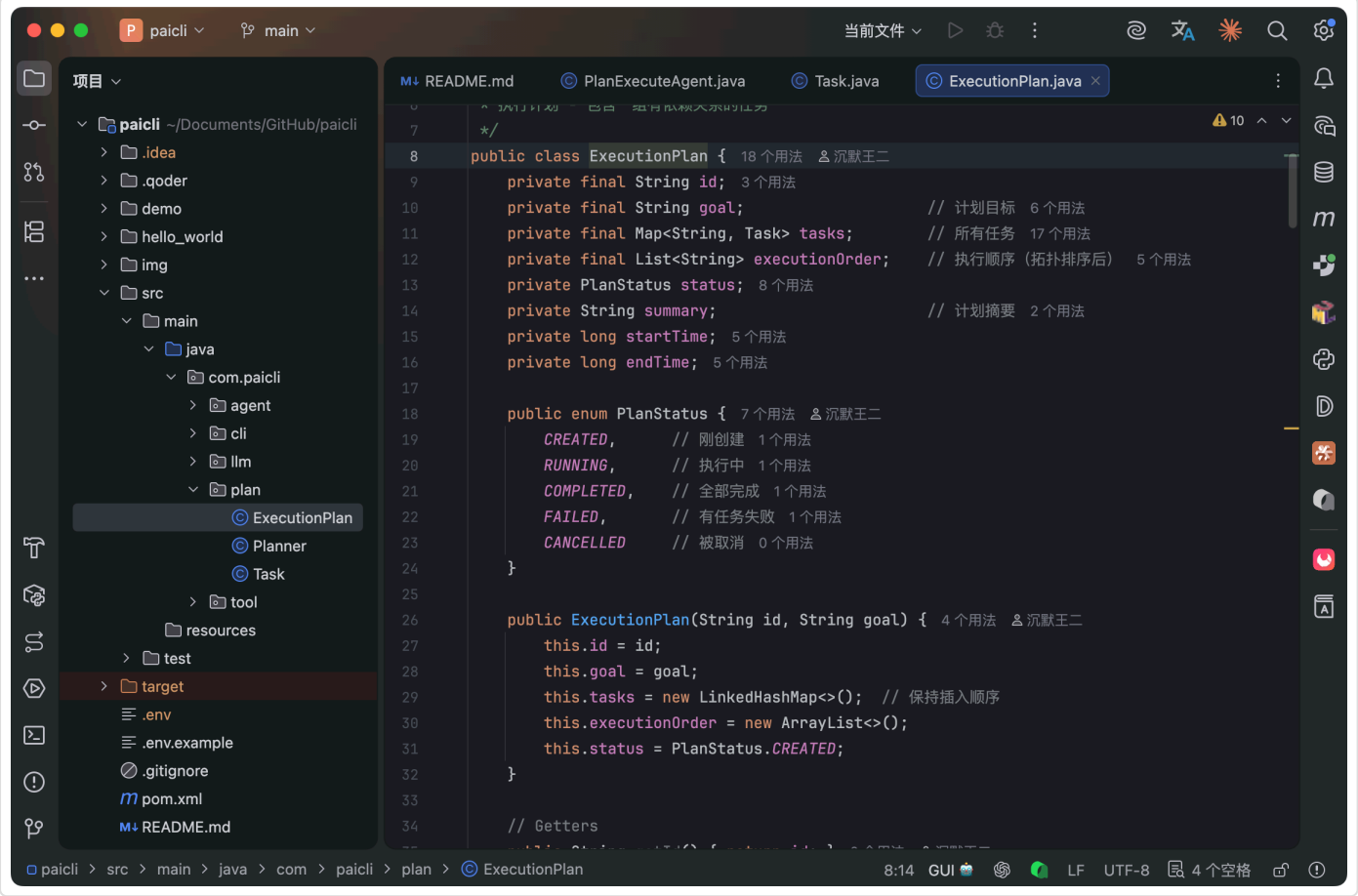
08、进阶：并行执行

并行执行可能遇到的问题

更智能的规划

规划的自我修正

PaiCLI 如何写到简历上？



拓扑排序算法

核心方法是 `computeExecutionOrder()`，使用拓扑排序算法把 DAG 转换成线性执行顺序。

拓扑排序的基本思想是：

- 找到所有入度为 0 的节点（没有依赖的任务）
- 把这些节点加入结果列表
- 移除这些节点及其出边
- 重复 1-3，直到所有节点都处理完

我们用 DFS 算法来实现：

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)

```
public boolean computeExecutionOrder() {
    executionOrder.clear();
    Set<String> visited = new HashSet<>();
    Set<String> visiting = new HashSet<>();

    for (Task task : tasks.values()) {
        if (!visited.contains(task.getId())) {
            if (!topologicalSort(task, visited, visiting)) {
                return false; // 有环
            }
        }
    }

    Collections.reverse(executionOrder);
    return true;
}

private boolean topologicalSort(Task task, Set<String> visited, Set<String> visiting) {
    String id = task.getId();

    if (visiting.contains(id)) {
        return false; // 有环，排序失败
    }
    if (visited.contains(id)) {
        return true;
    }

    visiting.add(id);

    // 递归处理所有依赖
    for (String depId : task.getDependencies()) {
        Task dep = tasks.get(depId);
        if (dep != null) {
            if (!topologicalSort(dep, visited, visiting)) {
                return false;
            }
        }
    }

    visiting.remove(id);
    visited.add(id);
    executionOrder.add(id);
    return true;
}
```

算法用两个集合来跟踪状态：**visiting** 是当前递归栈中的节点，用于检测环；**visited** 是已处理完的节点，用于避免重复处理。

如果检测到环 (**visiting.contains(id)**)，说明任务依赖关系有问题，比如 A 依赖 B，B 依赖 C，C 又依赖 A。这种情况下计划无法执行，需要报错提醒。

计划状态管理

执行计划本身也有状态：

- **CREATED**：刚创建，还没开始执行
- **RUNNING**：正在执行中
- **COMPLETED**：所有任务都完成
- **FAILED**：有任务失败
- **CANCELLED**：被取消

状态转换由执行结果决定：

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)

```
public void markStarted() {
    this.status = PlanStatus.RUNNING;
    this.startTime = System.currentTimeMillis();
}

public void markCompleted() {
    this.status = PlanStatus.COMPLETED;
    this.endTime = System.currentTimeMillis();
}

public boolean hasFailed() {
    return tasks.values().stream()
        .anyMatch(t -> t.getStatus() == TaskStatus.FAILED);
}
```

计划级别的状态让用户能快速了解整体执行情况，不需要逐个检查任务。

03、规划器实现

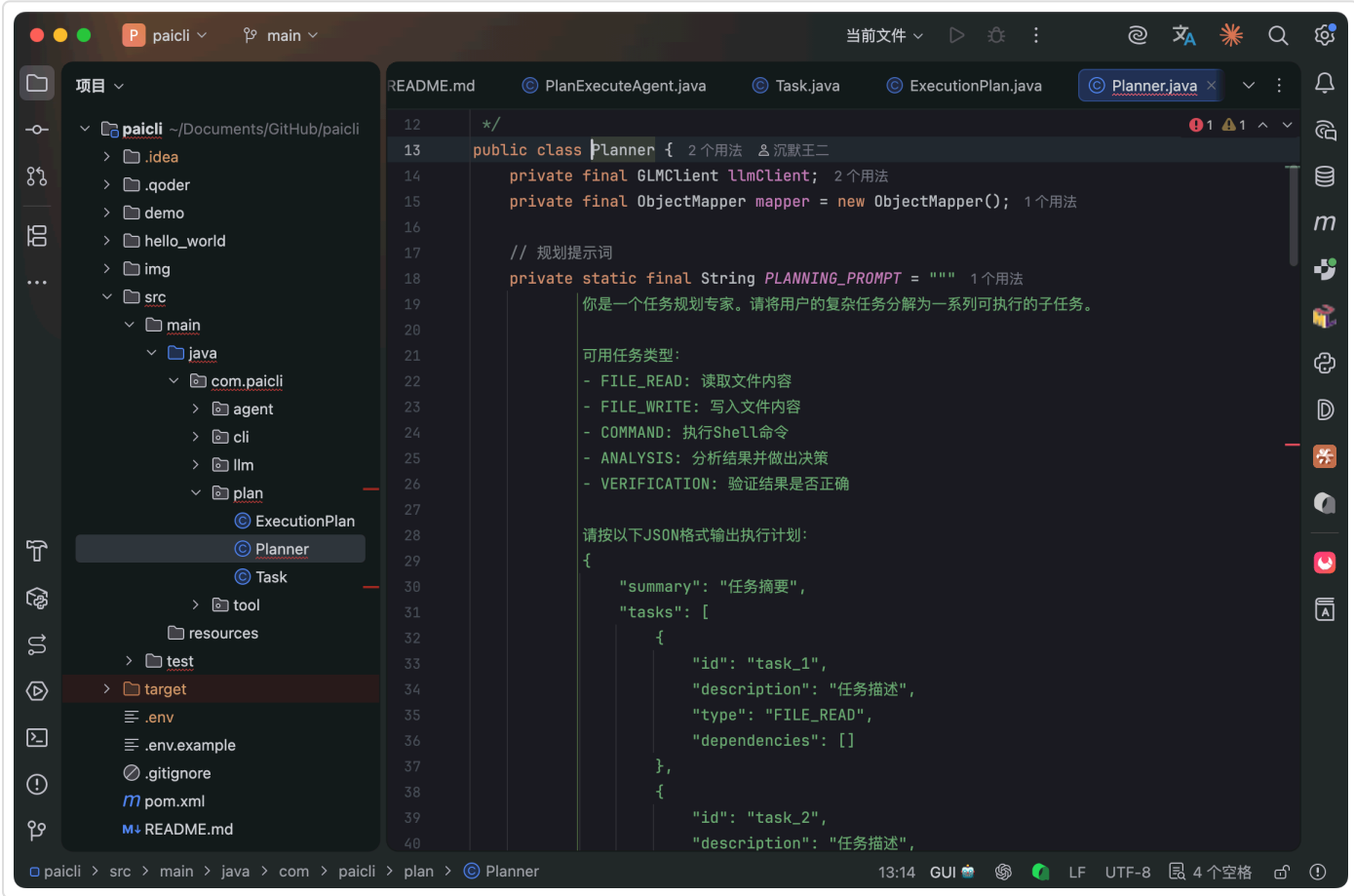
规划器负责把用户输入的复杂任务分解成可执行的计划。

```
public class Planner {
    private final GLMClient llmClient;

    public ExecutionPlan createPlan(String goal) throws IOException {
        // 1. 构建规划提示
        List<Message> messages = Arrays.asList(
            Message.system(PLANNING_PROMPT),
            Message.user("请为以下任务制定执行计划: \n" + goal)
        );

        // 2. 调用 LLM 生成计划
        ChatResponse response = llmClient.chat(messages, null);

        // 3. 解析 JSON 计划
        return parsePlan(goal, response.content());
    }
}
```



规划提示词工程

关键是给 LLM 一个清晰的提示，让它输出标准格式的计划。

提示词设计有几个原则：

第一，明确输出格式。告诉 LLM 必须输出 JSON，并且给出完整示例。

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)

CSS 复制代码

```
请按以下JSON格式输出执行计划：
{
  "summary": "任务摘要",
  "tasks": [
    {
      "id": "task_1",
      "description": "任务描述",
      "type": "FILE_READ",
      "dependencies": []
    }
  ]
}
```

第二，定义任务类型。列出所有可用的任务类型和用途，让 LLM 知道什么场景用什么类型。

diff 复制代码

```
可用任务类型：
- FILE_READ：读取文件内容，用于获取信息
- FILE_WRITE：写入文件内容，用于输出结果
- COMMAND：执行Shell命令，用于编译运行等
- ANALYSIS：分析结果，用于中间决策
- VERIFICATION：验证结果，用于检查正确性
```

第三，给出约束规则。明确任务的粒度、依赖关系的表达方式等。

markdown 复制代码

```
规则：
1. 每个任务必须有唯一的id (如 task_1, task_2)
2. dependencies列出依赖的任务id
3. 任务应该按执行顺序排列
4. 任务描述要具体明确
5. 复杂任务拆分为5-10个子任务
```

解析 LLM 输出

LLM 输出 JSON 后，我们需要解析并构建 Task 对象。这里有个细节：LLM 生成的任务 ID 可能重复或格式不统一，我们需要重新映射。

java 复制代码

```
private ExecutionPlan parsePlan(String goal, String planJson) throws IOException {
    // 清理可能的 markdown 代码块
    String cleaned = planJson.replaceAll("`json\\s*", "")
        .replaceAll("`\\s*", "")
        .trim();

    JsonNode root = mapper.readTree(cleaned);
    String summary = root.path("summary").asText();
    JsonNode tasksNode = root.path("tasks");

    ExecutionPlan plan = new ExecutionPlan(generatePlanId(), goal);
    plan.setSummary(summary);

    // 第一遍：创建任务，不处理依赖
    Map<String, String> idMapping = new HashMap<>();
    int taskIndex = 1;

    for (JsonNode taskNode : tasksNode) {
        String originalId = taskNode.path("id").asText();
        String newId = "task_" + taskIndex++;
        idMapping.put(originalId, newId);

        // 创建任务...
        Task task = new Task(newId, description, type);
        plan.addTask(task);
    }

    // 第二遍：处理依赖关系
    // ...
}
```

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

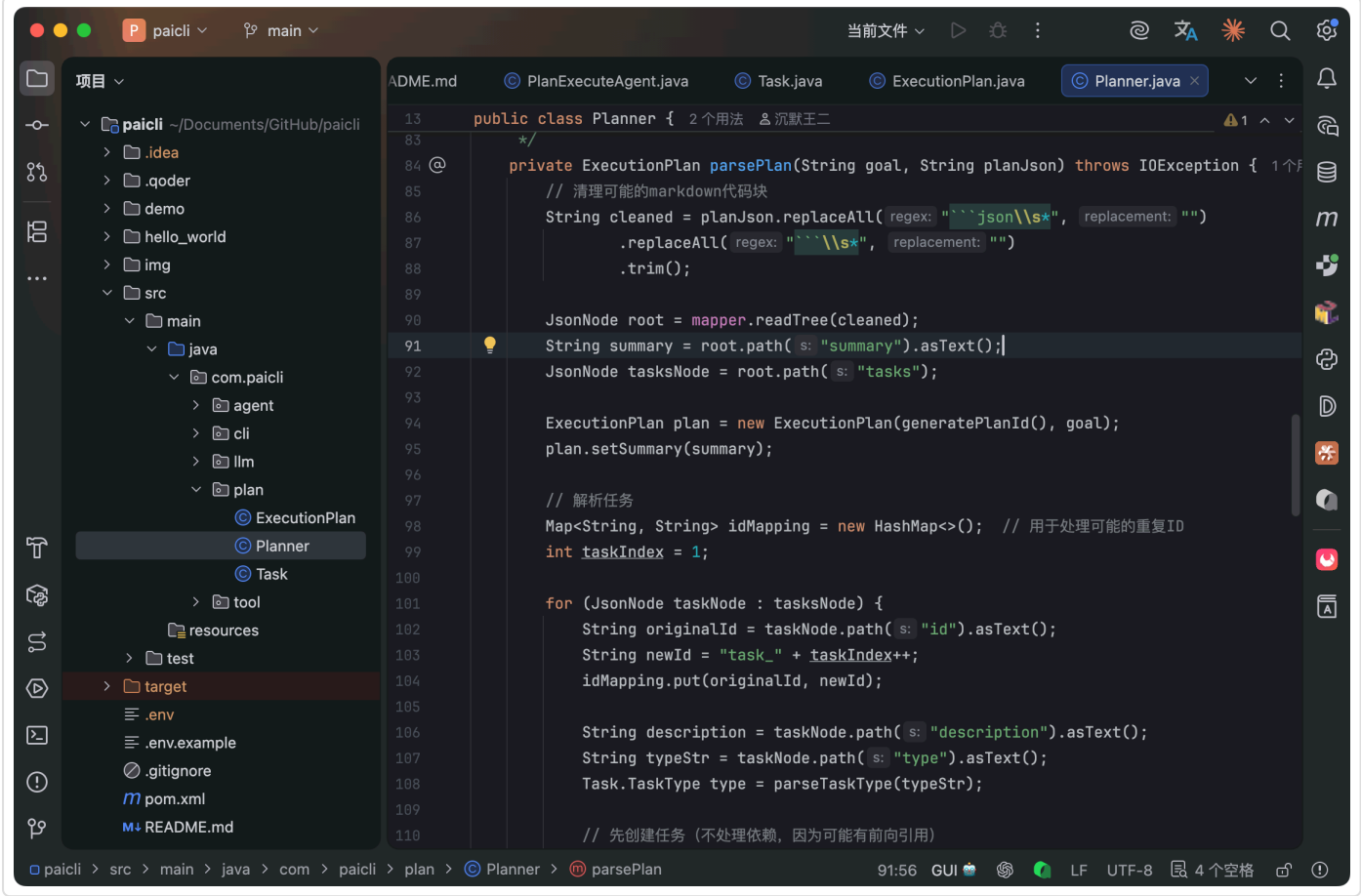
[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)



用两遍扫描的原因是：LLM 可能先定义 task_2，再定义 task_1，但 task_2 依赖 task_1。第一遍先创建所有任务，第二遍再建立依赖关系，避免前向引用问题。

重新规划

如果执行过程中某个步骤失败了，可以基于已完成的进度重新规划：

java 复制代码

```
public ExecutionPlan replan(ExecutionPlan failedPlan, String failureReason) {
    // 构建上下文：已完成任务 + 失败原因
    String context = buildContext(failedPlan, failureReason);

    // 重新生成计划
    return createPlan(context);
}
```

这样即使中途出错，也不用从头开始，已完成的任务可以保留。

04、PlanExecuteAgent

现在把规划器和执行器整合起来，实现 PlanExecuteAgent：

java 复制代码

```
public class PlanExecuteAgent {
    private final GLMClient llmClient;
    private final ToolRegistry toolRegistry;
    private final Planner planner;

    public String run(String userInput) {
        // 1. 创建执行计划
        ExecutionPlan plan = planner.createPlan(userInput);

        // 2. 显示计划
        System.out.println(plan.visualize());

        // 3. 执行计划
        for (String taskId : plan.getExecutionOrder()) {
            Task task = plan.getTask(taskId);
            executeTask(task);
        }

        // 4. 返回结果
        return buildResult(plan);
    }
}
```

智能模式切换

简单任务不需要规划。我们加一个启发式判断：

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

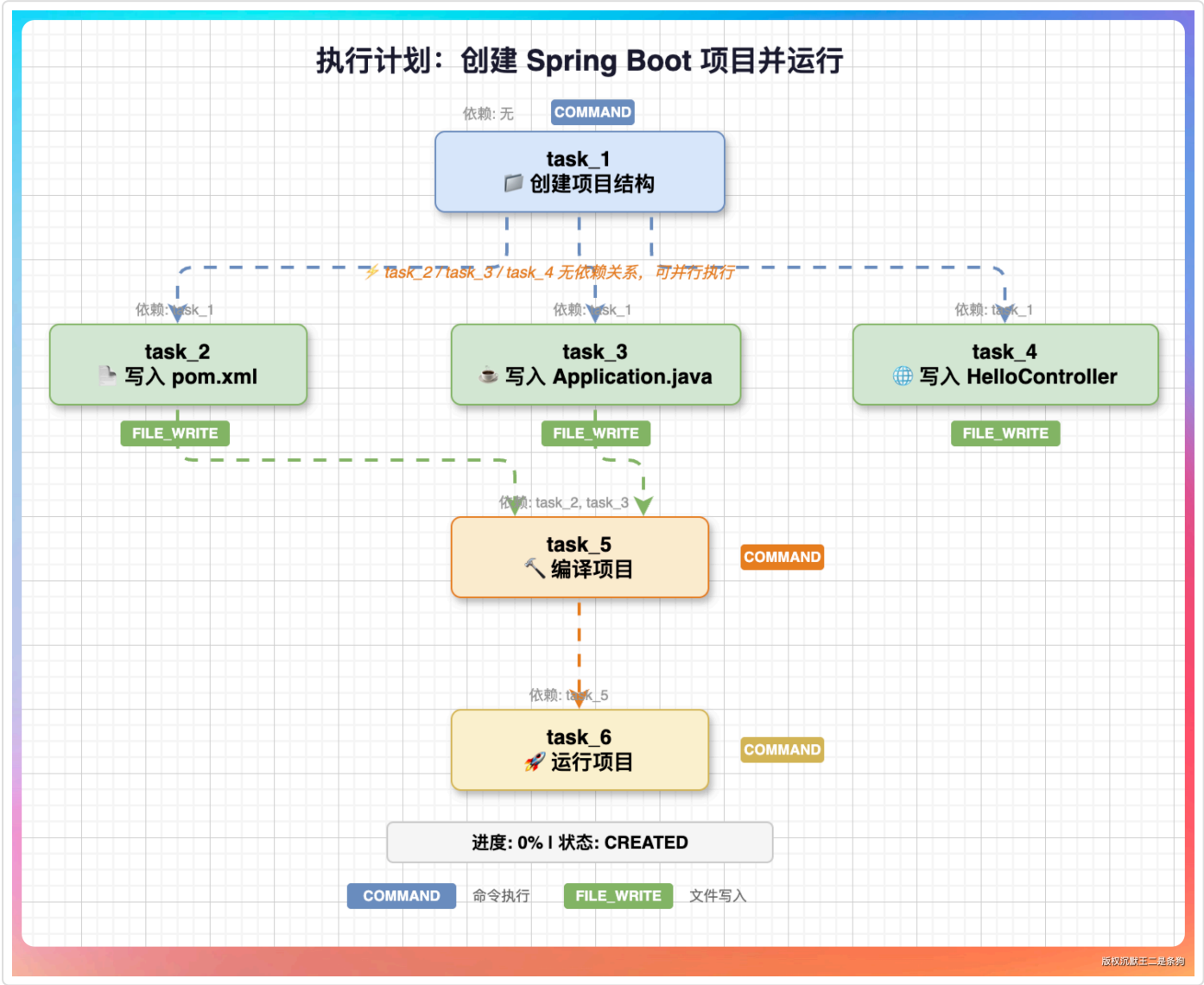
[PaiCLI 如何写到简历上？](#)

```
private boolean shouldPlan(String input) {
    // 包含多个动作关键词或长度超过50字符，需要规划
    String[] keywords = {"创建", "写", "读", "执行", "然后", "接着"};
    int actionCount = 0;
    for (String keyword : keywords) {
        if (input.contains(keyword)) actionCount++;
    }
    return actionCount >= 3 || input.length() > 50;
}
```

简单任务用 ReAct，复杂任务用 Plan-and-Execute，自动选择最优模式。

05、计划可视化

执行计划可以可视化展示，让用户清楚知道 Agent 要做什么：



执行过程中实时更新状态图标：🕒 → ▶ → ✅/❌

06、运行测试

编译运行：

```
mvn clean package
java -jar target/paicli-1.0-SNAPSHOT.jar
```

输入 `/plan` 进入计划模式，然后输入提示词：创建一个 Java 项目叫 demo，写一个 Hello 类输出 Hello World，然后编译运行



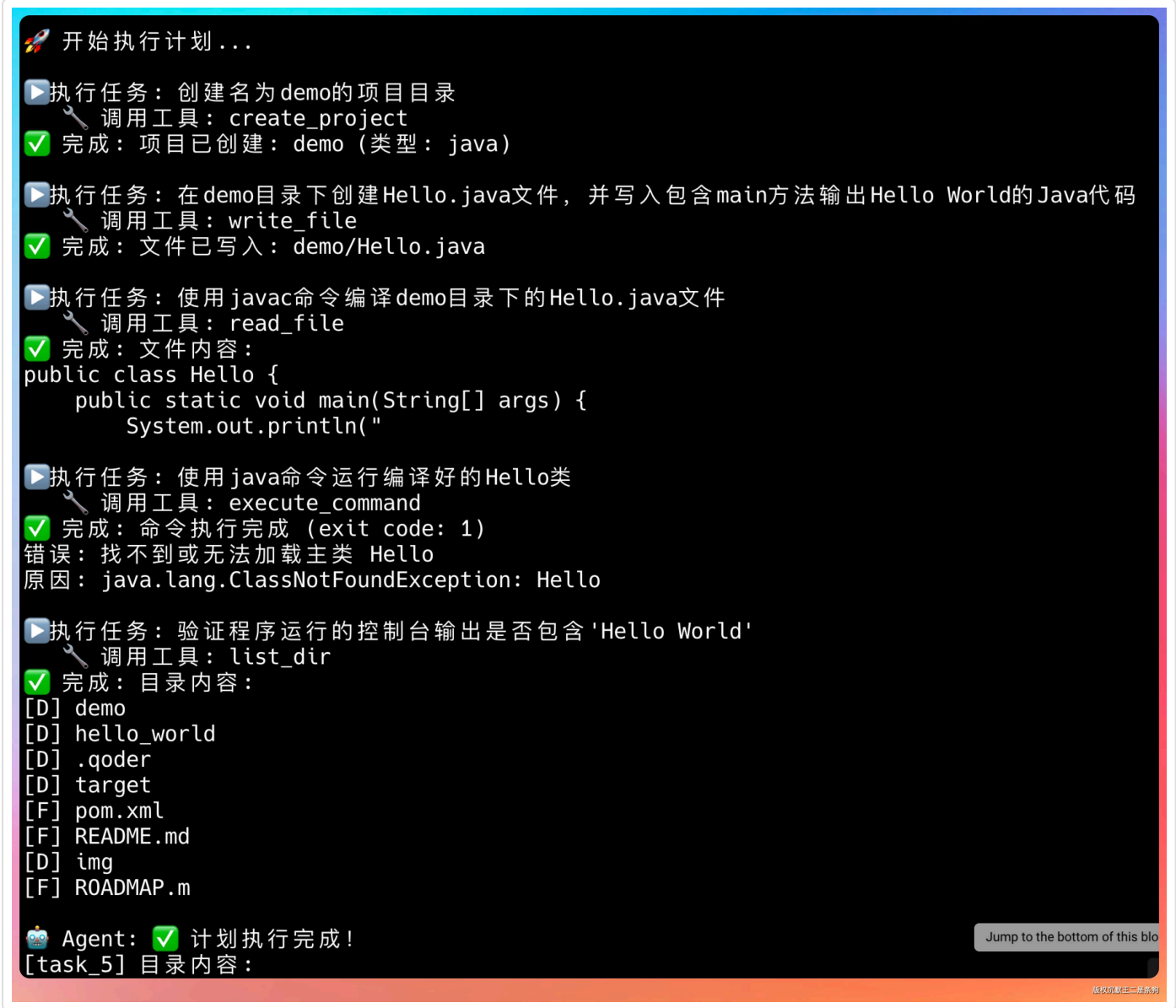
规划了 5 个任务，task5 依赖 task4，task4 依赖 task3，task3 依赖 task2，task2 依赖 task1。

目录

- 01、Plan-and-Execute 的核...
- 02、任务建模
 - 为什么需要任务建模
 - Task 类设计
 - 任务的生命周期
 - 依赖关系
 - 依赖关系
 - 执行计划
 - 拓扑排序算法
 - 计划状态管理
- 03、规划器实现
 - 规划提示词工程
 - 解析 LLM 输出
 - 重新规划
- 04、PlanExecuteAgent
 - 智能模式切换
- 05、计划可视化
- 06、运行测试
- 07、和 ReAct 的对比
 - 什么时候用 ReAct
 - 什么时候用 Plan-and-Execute
 - 混合使用
- 08、进阶：并行执行
 - 并行执行可能遇到的问题
 - 更智能的规划
 - 规划的自我修正
- PaiCLI 如何写到简历上？



然后开始执行计划。



这里我们可以加一个交互，看看用户是否有要补充的，是否要修改计划，然后再开始执行计划。

直接让 Codex 帮我们来补全这一步。

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

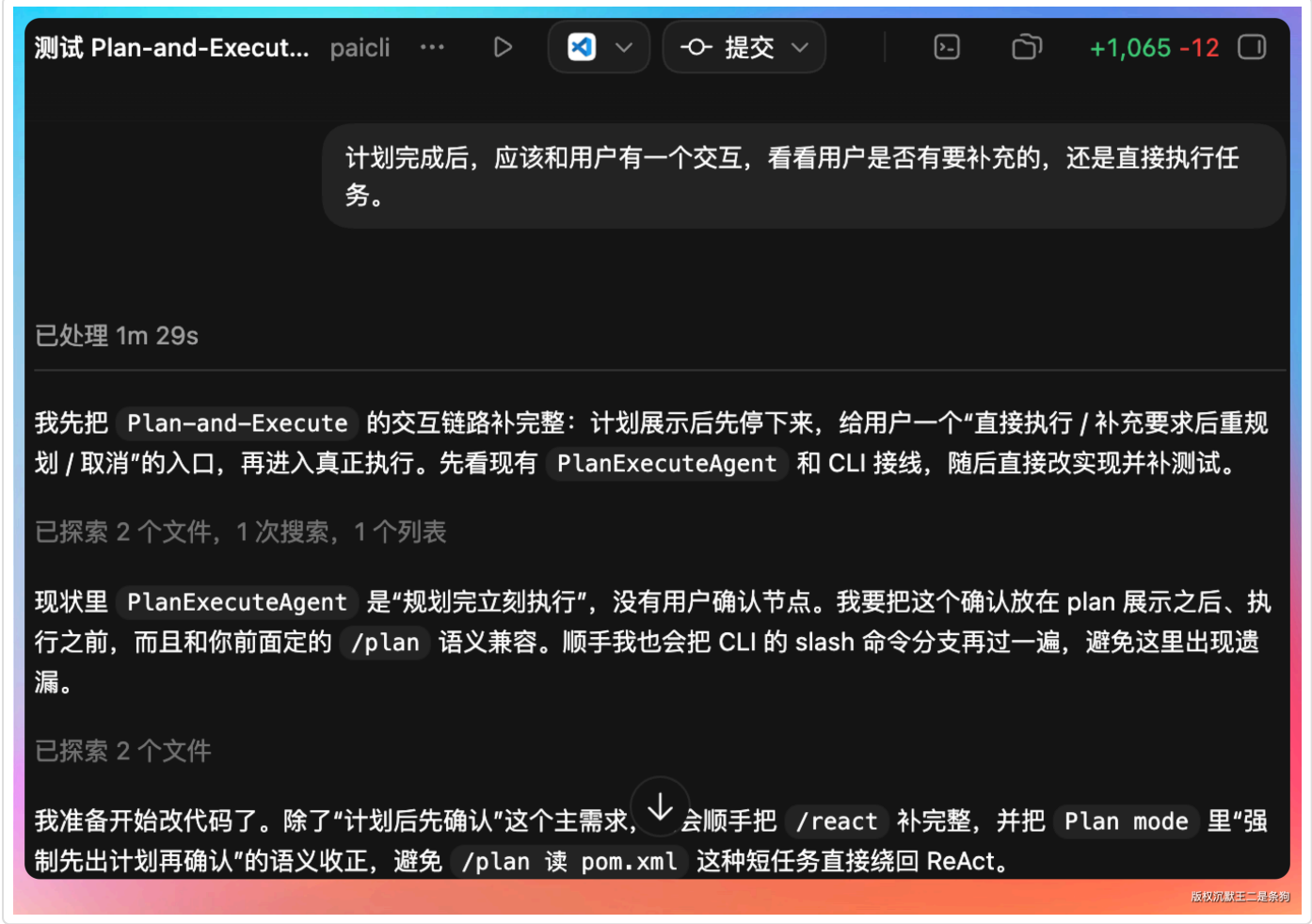
[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

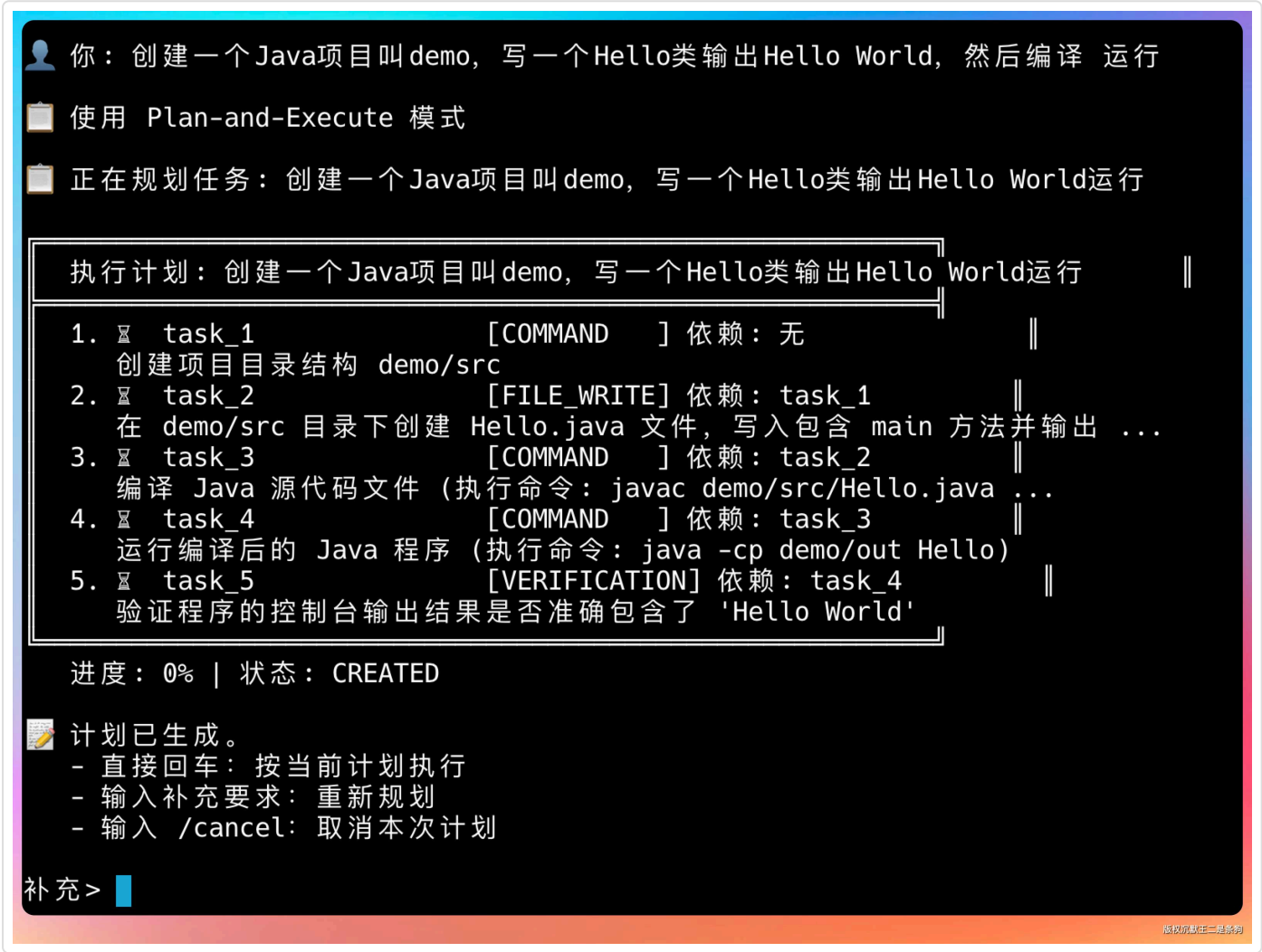
[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上?](#)



有了。



整个流程清晰可见，每一步都知道在做什么。

07、和 ReAct 的对比

两种模式各有优劣，适用场景不同。

特性	ReAct	Plan-and-Execute
LLM 调用次数	多（每步都调）	少（只调规划）
执行速度	慢（网络往返多）	快（本地执行多）
Token 消耗	高	低
灵活性	高（随时调整）	低（按 plan 执行）
可预测性	低	高
错误恢复	容易（随时改）	需要重规划

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)

特性	ReAct	Plan-and-Execute
适用场景	简单/探索性任务	复杂/确定性任务

什么时候用 ReAct

- 任务简单，1-3 步就能完成
- 需要探索性操作，不确定具体步骤
- 用户想看到思考过程
- 需要频繁的人机交互

比如“查看当前目录有什么文件”，直接 ReAct 一步完成。

什么时候用 Plan-and-Execute

- 任务复杂，需要多步操作
- 步骤之间有明确依赖关系
- 追求执行效率
- 需要可预测的执行流程

比如“搭建一个完整的 Web 项目，包括前后端、数据库、部署”，用 Plan-and-Execute 更合适。

混合使用

实际产品中，两种模式可以混合使用：

1. 用 Plan-and-Execute 制定整体计划

2. 每个任务内部用 ReAct 执行

3. 如果某步失败，用 ReAct 分析原因并决定是重试还是重规划

markdown 复制代码

这种混合模式兼顾了效率和灵活性，Claude Code 和 Codex 内部都是类似的架构。

08、进阶：并行执行

当前实现是顺序执行，但 DAG 中无依赖的任务可以并行。

```
// 获取所有可执行的任务（依赖都已完成）
List<Task> executableTasks = plan.getExecutableTasks();

// 并行执行
List<CompletableFuture<Void>> futures = executableTasks.stream()
    .map(task -> CompletableFuture.runAsync(() -> executeTask(task)))
    .toList();

// 等待全部完成
CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
```

java 复制代码

比如“创建项目”和“写 README”可以同时进行，进一步提升效率。

并行执行可能遇到的问题

并行执行可能会遇到这样的问题：

- **资源冲突**：两个任务同时写同一个文件，会导致数据丢失。
- **输出混乱**：两个任务的日志同时输出，用户看不清哪个是哪个。
- **错误处理复杂**：一个任务失败，其他正在执行的任务怎么办？

我们已经实现了哈。

提示词：

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)

markdown 复制代码

```
/plan 请把任务拆成可并行的 DAG:
1. 读取 pom.xml
2. 列出 src/main/java 下的文件
3. 列出 src/test/java 下的文件
4. 读取 README.md 的前 80 行
5. 最后汇总以上结果，告诉我这个项目的构建方式、源码结构和测试情况。
要求：前 4 个任务彼此独立并行执行，最后 1 个任务依赖前 4 个任务；在执行日志里明确提示哪些任务是并行执行的。
```

目录

01、Plan-and-Execute 的核...

02、任务建模

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

03、规划器实现

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

04、PlanExecuteAgent

[智能模式切换](#)

05、计划可视化

06、运行测试

07、和 ReAct 的对比

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

08、进阶：并行执行

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)

```
你： /plan 请把任务拆成可并行的 DAG:
1. 读取 pom.xml
2. 列出 src/main/java 下的文件
3. 列出 src/test/java 下的文件
4. 读取 README.md 的前 80 行
5. 最后汇总以上结果，告诉我这个项目的构建方式、源码结构和测试情况。
要求：前 4 个任务彼此独立并行执行，最后 1 个任务依赖前 4 个任务；在执行日志里明确提示哪些任务是并行执行的。
📁 使用 Plan-and-Execute 模式

📁 正在规划任务：请把任务拆成可并行的 DAG:

执行计划：请把任务拆成可并行的 DAG:

1. task_1 [FILE_READ ] 依赖：无
   并行分支1: 读取源数据文件内容
2. task_2 [FILE_READ ] 依赖：无
   并行分支2: 读取系统配置文件
3. task_3 [COMMAND ] 依赖：task_1,task_2
   结合源数据和配置，执行数据预处理和清洗脚本
4. task_4 [ANALYSIS ] 依赖：task_3
   对清洗后的数据进行深度业务逻辑分析并得出结论
5. task_5 [FILE_WRITE] 依赖：task_4
   将分析得出的结论写入到最终的报告文件中
6. task_6 [VERIFICATION] 依赖：task_5
   校验生成的报告文件，验证数据完整性与格式正确性

进度：0% | 状态：CREATED
```

开始并行执行任务。

```
🚀 开始执行计划 ...

< 本轮并行执行 2 个任务：task_1, task_2
▶ 并行任务 [task_1]: 并行分支1: 读取源数据文件内容
▶ 并行任务 [task_2]: 并行分支2: 读取系统配置文件
  🔧 调用工具：list_dir
  🔧 调用工具：read_file
  🔧 调用工具：read_file
  🔧 调用工具：read_file
✅ 完成 [task_1]: 目录内容：
[D] demo
[D] hello_world
[D] .qoder
[D] target
[F] pom.xml
[F] README.md
[D] img
[F] ROADMAP.m

✅ 完成 [task_2]: 读取文件失败：/etc/os-release
读取文件失败：/etc/hostname
文件内容：
#
# macOS Notice
#
# This file is not consulted

▶ 执行任务 [task_3]: 结合源数据和配置，执行数据预处理和清洗脚本
  🔧 调用工具：list_dir
  🔧 调用工具：list_dir
  🔧 调用工具：read_file
  🔧 调用工具：read_file
✅ 完成 [task_3]: 目录内容：
[D] demo
[D] hello world
```

当所有并行任务结束后，开始汇总。

目录内容：
[D] java

文件内容：
<?xml version="1.0" encoding="UTF-8"?>
<project

▶ 执行任务 [task_5]：将分析得出的结论写入到最终的报告中

🔧 调用工具：list_dir

✅ 完成 [task_5]：目录为空或不存在

▶ 执行任务 [task_6]：校验生成的报告文件，验证数据完整性与格式正确性

🔧 调用工具：execute_command

🔧 调用工具：execute_command

✅ 完成 [task_6]：命令执行完成 (exit code: 0)

/usr/bin/power_report.sh
/usr/libexec/gkreport
/usr/libexec/pkreporter
/usr/li

🤖 Agent：✅ 计划执行完成！

[task_6] 命令执行完成 (exit code: 0)

/usr/bin/power_report.sh
/usr/libexec/gkreport
/usr/libexec/pkreporter
/usr/libexec/signpost_reporter
/usr/libexec/rtcreportingd
/usr/libexec/logd_reporter
/usr/local/lib/node_modules/cnpm/node_modules/queue-microtask/README.md
/usr/local/lib/node_modules/cnpm/node_modules/is-docker/readme.md
/usr/local/lib/node_modules/cnpm/node_modules/osx-release/readme.md
/usr/local/lib/node_modules/cnpm/node_modules/digest-header/README.md

更智能的规划

当前的规划器只调用一次 LLM，生成完整计划。但复杂任务往往需要**分层规划**：

第一层规划：确定主要阶段

diff 复制代码

- 阶段1：环境搭建
- 阶段2：核心功能开发
- 阶段3：测试验证

第二层规划：细化每个阶段

- 阶段1包含：安装依赖、配置环境、初始化项目
- 阶段2包含：写模块A、写模块B、集成测试

分层规划的好处是：高层计划稳定，低层计划可以灵活调整。如果某个阶段的详细计划有问题，只需要重规划这个阶段，不影响整体。

规划的自我修正

LLM 制定的计划不一定完美，可能遗漏步骤、顺序错误、依赖不合理。我们需要**规划的自我修正**机制。

一种方法是**规划验证**：在执行前，用规则检查计划是否合理。

public List<String> validatePlan(ExecutionPlan plan) {
 List<String> errors = new ArrayList<>();

 // 检查是否有重复ID
 // 检查依赖是否存在
 // 检查是否有循环依赖
 // 检查任务类型是否合法

 return errors;
}

另一种方法是**规划反馈**：执行几步后，评估计划质量，必要时重新规划。

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)

java 复制代码

```
if (successRate < 0.5) {  
    // 成功率太低，重新规划  
    plan = planner.replan(plan, "前序任务成功率低");  
}
```

PaiCLI 如何写到简历上？

PaiCLI 项目（第 2 期） | 2026.04 - 2026.06 | Agent 开发

项目描述：为 Agent CLI 加入 Plan-and-Execute 能力，实现任务分解、DAG 依赖管理和拓扑排序执行。

技术栈：Java 17、Maven、GLM-5.1 API、DAG 拓扑排序、JSON 解析

核心职责：

- 设计 Task 任务模型，实现 6 种任务类型和 5 种状态流转，支持任务依赖双向追踪和 DAG 有向无环图表示
- 实现基于 DFS 的拓扑排序算法，将任务 DAG 转换为线性执行顺序，能自动检测循环依赖并报错，确保任务按顺序执行；并使用线程池并发执行无依赖的多项任务，相比串行执行效率大幅提升
- 开发 Planner 规划器，使用 LLM 将复杂任务分解为 5-10 个可执行子任务，通过 JSON 格式输出计划，实现 ID 映射和前向引用处理
- 实现 PlanExecuteAgent，支持根据任务复杂度自动切换 ReAct 和 Plan-and-Execute 两种模式
- 集成 JLine3 实现交互式命令行界面，支持命令历史（上下箭头）、Tab 自动补全、行编辑和语法高亮，用户体验接近原生 Shell



1人已点赞



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论

目录

[01、Plan-and-Execute 的核...](#)

[02、任务建模](#)

[为什么需要任务建模](#)

[Task 类设计](#)

[任务的生命周期](#)

[依赖关系](#)

[依赖关系](#)

[执行计划](#)

[拓扑排序算法](#)

[计划状态管理](#)

[03、规划器实现](#)

[规划提示词工程](#)

[解析 LLM 输出](#)

[重新规划](#)

[04、PlanExecuteAgent](#)

[智能模式切换](#)

[05、计划可视化](#)

[06、运行测试](#)

[07、和 ReAct 的对比](#)

[什么时候用 ReAct](#)

[什么时候用 Plan-and-Execute](#)

[混合使用](#)

[08、进阶：并行执行](#)

[并行执行可能遇到的问题](#)

[更智能的规划](#)

[规划的自我修正](#)

[PaiCLI 如何写到简历上？](#)